

# **LAPORAN PRAKTIKUM STRUKTUR DATA**

## **SINGLE LINKED LIST**

Disusun Oleh:

Ihsanul Zaky EL-Muhammady

NIM:2511533001

Dosen Pengampu:

Dr.Wahyudi S.T , M.T

Asisten Laboratorium:

Zahran Azaria Anvaya



**DEPARTEMEN INFORMATIKA**  
**FAKULTAS TEKNOLOGI INFORMASI**  
**UNIVERSITAS ANDALAS**  
**PADANG 2026**

## **KATA PENGANTAR**

Dengan penuh rasa syukur, saya mengucapkan terima kasih kepada Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga laporan praktikum “Struktur Data Linked List” ini dapat diselesaikan dengan baik.

Saya juga menyampaikan penghargaan kepada dosen pengampu Mata Kuliah Praktikum Struktur Data serta para asisten laboratorium yang telah memberikan arahan, bimbingan, dan bantuan selama proses praktikum berlangsung.

Laporan ini disusun sebagai salah satu bentuk dokumentasi kegiatan praktikum yang bertujuan untuk memahami konsep struktur data Linked List. Dalam laporan ini dibahas konsep dasar linked list, cara kerja hubungan antar node, serta penerapannya dalam pengelolaan data secara dinamis. Selain itu, dijelaskan pula beberapa operasi dasar pada linked list seperti penambahan node di awal, di akhir, maupun pada posisi tertentu, serta proses penelusuran data pada linked list.

Saya menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan agar laporan ini dapat menjadi lebih baik di masa yang akan datang, serta dapat memberikan manfaat dan menambah pemahaman mengenai struktur data linked list.

Padang, April 2026

Penulis

## DAFTAR ISI

|                                          |  |
|------------------------------------------|--|
| <b>KATA PENGANTAR.....</b>               |  |
| <b>DAFTAR ISI.....</b>                   |  |
| <b>BAB I PENDAHULUAN .....</b>           |  |
| 1.1 Latar Belakang .....                 |  |
| 1.2 Tujuan .....                         |  |
| 1.3 Manfaat.....                         |  |
| <b>BAB II PEMBAHASAN</b>                 |  |
| 2.1 Definisi Linked List Dalam Java..... |  |
| 2.2 NodeSLL_2511533001.....              |  |
| 2.3 TambahSLL_2511533001.....            |  |
| 2.4 PencarianSLL_2511533001.....         |  |
| 2.5 HapusSLL_2511533001 .....            |  |
| 2.6 Tugas .....                          |  |
| <b>BAB III KESIMPULAN .....</b>          |  |
| 3.1 Kesimpulan .....                     |  |
| 3.2 Saran.....                           |  |
| <b>DAFTAR PUSTAKA.....</b>               |  |

# **BAB I**

## **PENDAHULUAN**

### **1.1 Latar Belakang**

Dalam pengolahan data, terdapat kondisi di mana data perlu disusun dan dihubungkan secara dinamis agar lebih mudah dikelola. Jika dibandingkan dengan struktur data seperti Array atau ArrayList yang memiliki ukuran tetap dan penyimpanan data berurutan, Linked List lebih fleksibel karena setiap elemen disimpan dalam bentuk node yang saling terhubung melalui pointer. Dengan cara kerja ini, penambahan maupun penghapusan data dapat dilakukan tanpa perlu menggeser elemen lain.

Single Linked List (SLL) merupakan salah satu jenis linked list yang memiliki alur hubungan satu arah. Setiap node pada SLL terdiri dari data dan pointer yang menunjuk ke node berikutnya. Karena node tidak harus berada pada lokasi memori yang berurutan, struktur data ini lebih dinamis dalam pengelolaan data dibandingkan array. Selain itu, operasi seperti penambahan node di depan, di belakang, maupun pada posisi tertentu dapat dilakukan dengan lebih efisien selama referensi node diketahui.

Melalui praktikum ini, konsep Single Linked List dipelajari untuk memahami bagaimana node saling terhubung dan bagaimana manipulasi pointer dilakukan dalam proses pengolahan data. Pada praktikum ini juga diterapkan beberapa operasi dasar seperti insert di depan, insert di belakang, insert pada posisi tertentu, traversal, searching, serta delete node. Dengan adanya penerapan tersebut, praktikum membantu dalam memahami cara kerja linked list secara lebih terstruktur dan sistematis.

### **1.2 Tujuan**

1. Memahami konsep dasar struktur data Single Linked List (SLL).
2. Mengetahui perbedaan antara linked list dengan array dalam pengelolaan data.
3. Memahami cara kerja node dan hubungan antar node pada linked list.
4. Memahami operasi dasar pada Single Linked List seperti insert, delete, traversal, dan searching.
5. Mengembangkan logika pemrograman dalam mengelola data menggunakan struktur data linked list.

### **1.3 Manfaat**

1. Meningkatkan pemahaman tentang struktur data Single Linked List dalam pemrograman.
2. Melatih kemampuan berpikir logis dan sistematis dalam pengelolaan data dinamis.
3. Membantu memahami proses manipulasi data seperti penambahan dan penghapusan node.

4. Memberikan gambaran tentang cara kerja hubungan antar node pada linked list.
5. Menambah pengalaman dalam menerapkan struktur data linked list pada pemrograman.

## BAB II

### PEMBAHASAN

#### 2.1 Definisi Linked List Dalam Java

Linked List merupakan salah satu struktur data linier yang digunakan untuk menyimpan sekumpulan data secara dinamis. Pada Linked List, data disimpan di dalam node, di mana setiap node memiliki dua bagian utama yaitu data dan referensi (pointer) yang menunjuk ke node berikutnya. Berbeda dengan array yang penyimpanan datanya berurutan di memori, Linked List menyimpan node secara terpisah namun tetap saling terhubung melalui pointer.

Dalam Java, Linked List dapat dibuat secara manual menggunakan class node maupun menggunakan class bawaan LinkedList dari Java Collections Framework. Struktur data ini memiliki kelebihan dalam proses penambahan dan penghapusan data karena tidak memerlukan pergeseran elemen seperti pada array. Oleh karena itu, Linked List cocok digunakan untuk pengolahan data yang bersifat dinamis dan sering mengalami perubahan ukuran.

Single Linked List (SLL) adalah jenis linked list yang setiap node hanya memiliki satu pointer yang mengarah ke node berikutnya. Operasi dasar yang umum digunakan pada linked list antara lain insert, delete, traversal, dan searching. Dengan memahami Linked List, proses pengelolaan data dalam pemrograman dapat dilakukan dengan lebih fleksibel dan efisien.

#### 2.2 NodeSLL\_2511533001

```
1 package pekan5_2511533001;
2
3 public class NodeSLL_2511533001 {
4     // node bagian data
5     int data_3001;
6     // Pointer ke node berikutnya
7     NodeSLL_2511533001 next_3001;
8     // Konstruktor menginisialisasi node dengan data
9     public NodeSLL_2511533001(int data_3001)
10    {
11        this.data_3001 = data_3001;
12        this.next_3001 = null;
13    }
14 }
```

Penjelasan Program:

Program di atas merupakan class NodeSLL\_2511533001 yang digunakan sebagai node pada Single Linked List (SLL). Class ini memiliki atribut data\_3001 untuk menyimpan data bertipe integer dan next\_3001 sebagai pointer yang menghubungkan node ke node berikutnya.

Selain itu, terdapat constructor NodeSLL\_2511533001(int data\_3001) yang berfungsi untuk menginisialisasi nilai data pada node dan mengatur next\_3001 menjadi null. Class node ini menjadi dasar dalam pembentukan dan pengelolaan struktur data linked list.

## 2.3 TambahSLL\_2511533001

```
1 package pekan5_2511533001;
2
3 class NodeSLL_2511533001{
4     int data_3001;
5     NodeSLL_2511533001 next_3001;
6
7     NodeSLL_2511533001(int data_3001) {}
8     this.data_3001 = data_3001;
9     this.next_3001 = null;
10 }
11
12
13 public class TambahSLL_2511533001 {
14
15     // tambah di depan
16     public static NodeSLL_2511533001 insertAtFront_3001(NodeSLL_2511533001 head_3001, int value_3001) {
17         NodeSLL_2511533001 new_node_3001 = new NodeSLL_2511533001(value_3001);
18         new_node_3001.next_3001 = head_3001;
19         return new_node_3001;
20     }
21
22     // tambah di belakang
23     public static NodeSLL_2511533001 insertAtEnd_3001(NodeSLL_2511533001 head_3001, int value_3001) {
24         NodeSLL_2511533001 new_node_3001 = new NodeSLL_2511533001(value_3001);
25
26         if (head_3001 == null)
27             return new_node_3001;
28
29         NodeSLL_2511533001 temp_3001 = head_3001;
30         while (temp_3001.next_3001 != null) {
31             temp_3001 = temp_3001.next_3001;
32         }
33
34         temp_3001.next_3001 = new_node_3001;
35         return head_3001;
36     }
37
38     // tambah di posisi (FIX sesuai modul)
39     static NodeSLL_2511533001 insertPos_3001(NodeSLL_2511533001 head_3001, int position_3001, int value_3001) {
40
41         if (position_3001 < 1) {
42             System.out.println("Invalid position");
43             return head_3001;
44         }
45
46         // jika di depan
47         if (position_3001 == 1) {
48             NodeSLL_2511533001 new_node_3001 = new NodeSLL_2511533001(value_3001);
49             new_node_3001.next_3001 = head_3001;
50             return new_node_3001;
51         }
52
53         NodeSLL_2511533001 temp_3001 = head_3001;
54
55         // maju ke node sebelum posisi
56         for (int i = 1; i < position_3001 - 1; i++) {
57             if (temp_3001 == null) {
58                 System.out.println("Posisi di luar jangkauan");
59                 return head_3001;
60             }
61             temp_3001 = temp_3001.next_3001;
62         }
63
64         NodeSLL_2511533001 new_node_3001 = new NodeSLL_2511533001(value_3001);
65
66         // bagian inti (yang sebelumnya salah di kode kamu)
67         new_node_3001.next_3001 = temp_3001.next_3001;
68         temp_3001.next_3001 = new_node_3001;
69
70         return head_3001;
71     }
72 }
73
74 // print list
75 public static void printList_3001(NodeSLL_2511533001 head_3001) {
76     NodeSLL_2511533001 curr_3001 = head_3001;
77
78     while (curr_3001 != null) {
79         System.out.print(curr_3001.data_3001);
80         if (curr_3001.next_3001 != null) {
81             System.out.print("-->");
82             curr_3001 = curr_3001.next_3001;
83         }
84         System.out.println();
85     }
86 }
87
88 public static void main(String[] args) {
89
90     // list awal: 2->3->5->6
91     NodeSLL_2511533001 head_3001 = new NodeSLL_2511533001(2);
92     head_3001.next_3001 = new NodeSLL_2511533001(3);
93     head_3001.next_3001.next_3001 = new NodeSLL_2511533001(5);
94     head_3001.next_3001.next_3001.next_3001 = new NodeSLL_2511533001(6);
95
96     System.out.print("Senarai berantai awal: ");
97     printList_3001(head_3001);
98
99     // tambah depan
100     System.out.print("tambah 1 simpul di depan: ");
101     head_3001 = insertAtFront_3001(head_3001, 1);
102     printList_3001(head_3001);
103
104     // tambah belakang
105     System.out.print("tambah 1 simpul di belakang: ");
106     head_3001 = insertAtEnd_3001(head_3001, 7);
107
108     printList_3001(head_3001);
109
110     // tambah posisi (4 di posisi ke-4)
111     System.out.print("tambah 1 simpul ke data 4: ");
112     head_3001 = insertPos_3001(head_3001, 4, 4);
113     printList_3001(head_3001);
114 }
```

### Penjelasan Program :

1. `class NodeSLL_2511533001` → Class ini digunakan sebagai node pada struktur data Single Linked List (SLL).
2. `int data_3001;` dan `NodeSLL_2511533001 next_3001;` → Variabel `data_3001` digunakan untuk menyimpan data, sedangkan `next_3001` berfungsi sebagai pointer ke node berikutnya.
3. `NodeSLL_2511533001(int data_3001)` → Constructor digunakan untuk menginisialisasi node baru dengan data tertentu dan mengatur pointer menjadi null.
4. `insertAtFront_3001()` → Method ini digunakan untuk menambahkan node baru di bagian depan linked list.
5. `insertAtEnd_3001()` → Method ini digunakan untuk menambahkan node baru di bagian akhir linked list dengan cara menelusuri node sampai ke bagian terakhir.
6. `insertPos_3001()` → Method ini digunakan untuk menambahkan node pada posisi tertentu di dalam linked list.
7. `newNode_3001.next_3001 = temp_3001.next_3001;` → Pointer node baru diarahkan ke node berikutnya agar hubungan linked list tetap tersambung.
8. `temp_3001.next_3001 = newNode_3001;` → Node baru dimasukkan ke posisi yang diinginkan pada linked list.
9. `printList_3001()` → Method ini digunakan untuk menampilkan seluruh isi linked list dari depan ke belakang menggunakan proses traversal.
10. Pada `main()` dibuat linked list awal 2 -> 3 -> 5 -> 6, kemudian dilakukan penambahan data di depan, belakang, dan posisi tertentu sehingga hasil akhirnya menjadi 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7.

### Output :

```
Senarai berantai awal: 2-->3-->5-->6
tambah 1 simpul di depan: 1-->2-->3-->5-->6
tambah 1 simpul di belakang: 1-->2-->3-->5-->6-->7
tambah 1 simpul ke data 4: 1-->2-->3-->4-->5-->6-->7
```



## 2.4 PencarianSLL\_2511533001

```
1 package pekan2_2511533001;
2
3 public class PencarianSLL_2511533001 {
4     static boolean searchKey_3001 (NodeSLL_2511533001 head_3001, int key_3001) {
5         NodeSLL_2511533001 curr_3001 = head_3001;
6         while (curr_3001 != null) {
7             if (curr_3001.data_3001 == key_3001 )
8                 return true;
9             curr_3001 = curr_3001.next_3001; }
10        return false; }
11    public static void traversal_3001 (NodeSLL_2511533001 head_3001) {
12        NodeSLL_2511533001 curr_3001 = head_3001;
13        while (curr_3001 != null) {
14            System.out.print(" " + curr_3001.data_3001);
15            curr_3001 = curr_3001.next_3001;
16        }
17        System.out.println();
18    }
19    public static void main(String[] args) {
20        NodeSLL_2511533001 head_3001 = new NodeSLL_2511533001(14);
21        head_3001.next_3001 = new NodeSLL_2511533001(21);
22        head_3001.next_3001.next_3001 = new NodeSLL_2511533001(13);
23        head_3001.next_3001.next_3001.next_3001 = new NodeSLL_2511533001(30);
24        head_3001.next_3001.next_3001.next_3001.next_3001 = new NodeSLL_2511533001(10);
25        System.out.print("Penelusuran SLL: ");
26        traversal_3001(head_3001);
27        int key_3001 = 30;
28        System.out.print("cari data " + key_3001 + " = ");
29        if (searchKey_3001 (head_3001, key_3001))
30            System.out.println("ketemu");
31        else
32            System.out.println("tidak ada");
33    }}
```

### Penjelasan Program:

1. searchKey\_3001() → Method ini digunakan untuk mencari data tertentu pada Single Linked List (SLL).
2. NodeSLL\_2511533001 curr\_3001 = head\_3001; → Variabel curr\_3001 digunakan sebagai pointer untuk menelusuri linked list mulai dari head.
3. while (curr\_3001 != null) → Program melakukan traversal node satu per satu sampai mencapai node terakhir.
4. if (curr\_3001.data\_3001 == key\_3001) → Program membandingkan data pada node dengan data yang dicari.
5. return true; → Jika data ditemukan, method akan mengembalikan nilai true.
6. curr\_3001 = curr\_3001.next\_3001; → Pointer digeser ke node berikutnya untuk melanjutkan pencarian data.
7. return false; → Jika seluruh node sudah dicek tetapi data tidak ditemukan, maka method mengembalikan nilai false.
8. traversal\_3001() → Method ini digunakan untuk menampilkan seluruh isi linked list dari depan ke belakang.
9. Pada main() dibuat linked list dengan data 14 -> 21 -> 13 -> 30 -> 10, kemudian method traversal\_3001() dipanggil untuk menampilkan isi linked list.
10. Program melakukan pencarian data 30 menggunakan searchKey\_3001() dan hasilnya menampilkan "ketemu" karena data tersebut ada di dalam linked list.

Output:

```
Penelusuran SSL: 14 21 13 30 10
cari data 30 = ketemu
```

## 2.5 HapusSLL\_2511533001

```
1 package pekan5_2511533001;
2
3 public class HapusSLL_2511533001 {
4     public static NodeSLL_2511533001 deleteHead_3001(NodeSLL_2511533001 head_3001) {
5         if (head_3001 == null)
6             return null;
7         head_3001 = head_3001.next_3001;
8         return head_3001;
9     }
10
11     public static NodeSLL_2511533001 removeLastNode_3001 (NodeSLL_2511533001 head_3001) {
12         if (head_3001 == null) {
13             return null;
14         }
15         if (head_3001.next_3001 == null) {
16             return null;
17         }
18         NodeSLL_2511533001 secondLast_3001 = head_3001;
19         while (secondLast_3001.next_3001.next_3001 != null) {
20             secondLast_3001 = secondLast_3001.next_3001;
21         }
22         secondLast_3001.next_3001 = null;
23         return head_3001;
24     }
25
26     public static NodeSLL_2511533001 deleteNode_3001 (NodeSLL_2511533001 head_3001, int position_3001) {
27         NodeSLL_2511533001 temp_3001 = head_3001;
28         NodeSLL_2511533001 prev_3001 = null;
29         if (temp_3001 == null)
30             return head_3001;
31         if (position_3001 == 1) {
32             head_3001 = temp_3001.next_3001;
33             return head_3001;
34         }
35         for (int i = 1; temp_3001 != null && i < position_3001; i++) {
36             prev_3001 = temp_3001;
37             temp_3001 = temp_3001.next_3001;
38         }
39         if (temp_3001 != null) {
40             prev_3001.next_3001 = temp_3001.next_3001;
41         }
42         else {
43             System.out.println("Data tidak ada");
44             return head_3001;
45         }
46         return prev_3001;
47     }
48     public static void printList_3001 (NodeSLL_2511533001 head_3001) {
49         NodeSLL_2511533001 curr_3001 = head_3001;
50         while (curr_3001.next_3001 != null) {
51             System.out.print(curr_3001.data_3001+"-->");
52             curr_3001 = curr_3001.next_3001;
53         }
54         if (curr_3001.next_3001 == null) {
55             System.out.print(curr_3001.data_3001);
56             System.out.println();
57         }
58     }
59
60     public static void main(String[] args) {
61         NodeSLL_2511533001 head_3001 = new NodeSLL_2511533001(1);
62         head_3001.next_3001 = new NodeSLL_2511533001(2);
63         head_3001.next_3001.next_3001 = new NodeSLL_2511533001(3);
64         head_3001.next_3001.next_3001.next_3001 = new NodeSLL_2511533001(4);
65         head_3001.next_3001.next_3001.next_3001.next_3001 = new NodeSLL_2511533001(5);
66         head_3001.next_3001.next_3001.next_3001.next_3001.next_3001 = new NodeSLL_2511533001(6);
67         System.out.println("list awal: ");
68         printList_3001(head_3001);
69         head_3001 = deleteHead_3001(head_3001);
70         System.out.println("List setelah head dihapus: ");
71
72         printList_3001(head_3001);
73         head_3001 = removeLastNode_3001(head_3001);
74         System.out.println("List setelah simpul terakhir dihapus: ");
75         printList_3001(head_3001);
76         int position_3001 = 2;
77         head_3001 = deleteNode_3001(head_3001, position_3001);
78         System.out.println("List setelah posisi 2 dihapus: ");
79         printList_3001(head_3001);
80     }
81 }
82 }
```

Penjelasan Program:

1. deleteHead\_3001() → Method ini digunakan untuk menghapus node paling depan (head) pada Single Linked List.
2. if (head\_3001 == null) → Program mengecek apakah linked list kosong. Jika kosong, method langsung mengembalikan null.

3. `head_3001 = head_3001.next_3001;` → Head dipindahkan ke node berikutnya sehingga node pertama terhapus dari linked list.
4. `removeLastNode_3001()` → Method ini digunakan untuk menghapus node paling belakang pada linked list.
5. `if (head_3001.next_3001 == null)` → Jika linked list hanya memiliki satu node, maka setelah dihapus linked list menjadi kosong.
6. `while (secondLast_3001.next_3001.next_3001 != null)` → Program menelusuri linked list sampai menemukan node sebelum node terakhir.
7. `secondLast_3001.next_3001 = null;` → Pointer node sebelum terakhir diubah menjadi null sehingga node terakhir terhapus.
8. `deleteNode_3001()` → Method ini digunakan untuk menghapus node pada posisi tertentu.
9. `if (position_3001 == 1)` → Jika posisi yang dihapus adalah posisi pertama, maka proses penghapusan dilakukan pada head.
10. `for (int i = 1; temp_3001 != null && i < position_3001; i++)` → Program melakukan traversal untuk mencari node pada posisi yang akan dihapus.
11. `prev_3001.next_3001 = temp_3001.next_3001;` → Node pada posisi tertentu dihapus dengan menghubungkan node sebelumnya ke node setelahnya.
12. `printList_3001()` → Method ini digunakan untuk menampilkan seluruh isi linked list dari depan ke belakang.
13. Pada `main()` dibuat linked list 1 -> 2 -> 3 -> 4 -> 5 -> 6, kemudian dilakukan penghapusan head, penghapusan node terakhir, dan penghapusan node pada posisi ke-2 sehingga isi linked list berubah sesuai operasi yang dilakukan.

Output:

```
list awal:
1-->2-->3-->4-->5-->6
List setelah head dihapus:
2-->3-->4-->5-->6
List setelah simpul terakhir dihapus:
2-->3-->4-->5
List setelah posisi 2 dihapus:
2-->4-->5
```

## 2.6 Tugas

### 1. Pasien\_2511533001

```
package pekan5_2511533001;

public class Pasien_2511533001 {

    // data pasien
    private String namaPasien_3001;
    private String penyakit_3001;
    private int nomorAntrian_3001;

    // pointer ke node berikutnya
    Pasien_2511533001 next_3001;

    // konstruktor
    public Pasien_2511533001(String namaPasien_3001,
                               String penyakit_3001,
                               int nomorAntrian_3001) {

        this.namaPasien_3001 = namaPasien_3001;
        this.penakit_3001 = penyakit_3001;
        this.nomorAntrian_3001 = nomorAntrian_3001;

        this.next_3001 = null;
    }

    // getter nama pasien
    public String getNamaPasien_3001() {
        return namaPasien_3001;
    }

    // getter penyakit
    public String getPenyakit_3001() {
        return penyakit_3001;
    }

    // getter nomor antrian
    public int getNomorAntrian_3001() {
        return nomorAntrian_3001;
    }
}
```

#### Penjelasan Program :

1. class Pasien\_2511533001 → Class ini digunakan sebagai node pada struktur data Single Linked List untuk menyimpan data pasien rumah sakit.
2. namaPasien\_3001, penyakit\_3001, dan nomorAntrian\_3001 → Variabel ini digunakan untuk menyimpan nama pasien, keluhan/penyakit, dan nomor antrian pasien.
3. Pasien\_2511533001 next\_3001; → Variabel ini berfungsi sebagai pointer yang menghubungkan node pasien ke node berikutnya.
4. public Pasien\_2511533001(...) → Constructor digunakan untuk menginisialisasi data pasien saat objek dibuat.
5. this.namaPasien\_3001 = namaPasien\_3001; → Digunakan untuk mengisi nama pasien ke dalam node.
6. this.penakit\_3001 = penyakit\_3001; → Digunakan untuk menyimpan data keluhan atau penyakit pasien.
7. this.nomorAntrian\_3001 = nomorAntrian\_3001; → Digunakan untuk menyimpan nomor antrian pasien.
8. this.next\_3001 = null; → Pointer diatur menjadi null karena node baru belum terhubung dengan node lain.

9. `getNamaPasien_3001()` → Method getter yang digunakan untuk mengambil data nama pasien.
10. `getPenyakit_3001()` dan `getNomorAntrian_3001()` → Method getter yang digunakan untuk mengambil data penyakit dan nomor antrian pasien sehingga dapat digunakan pada program `RumahSakit_2511533001`.

## 2. RumahSakit\_2511533001

```

1 package pekan5_2511533001;
2 import java.util.Scanner;
3
4 public class RumahSakit_2511533001 {
5     Pasien_2511533001 head_3001, tail_3001;
6     int nomor_3001 = 0, jumlah_3001 = 0;
7
8     // insert pasien
9     void daftarkanPasien_3001(String nama_3001, String keluhan_3001) {
10
11         nomor_3001++;
12
13         Pasien_2511533001 baru_3001 =
14             new Pasien_2511533001(nama_3001, keluhan_3001, nomor_3001);
15
16         if (head_3001 == null) {
17             head_3001 = tail_3001 = baru_3001;
18         } else {
19             tail_3001.next_3001 = baru_3001;
20             tail_3001 = baru_3001;
21         }
22         jumlah_3001++;
23         System.out.println("Pasien berhasil didaftarkan!");
24     }
25
26     // delete head
27     void panggilPasien_3001() {
28
29         if (head_3001 == null) {
30             System.out.println("Antrian kosong.");
31             return;
32         }
33
34         System.out.println("\n== Pasien Dipanggil ==");
35         System.out.println("No Antrian : " + head_3001.getNomorAntrian_3001());
36         System.out.println("Nama : " + head_3001.getNamaPasien_3001());
37         System.out.println("Keluhan : " + head_3001.getPenyakit_3001());
38
39         head_3001 = head_3001.next_3001;
40
41         if (head_3001 == null) {
42             tail_3001 = null;
43         }
44         jumlah_3001--;
45     }
46
47     // display
48     void tampilkanAntrian_3001() {
49
50         if (head_3001 == null) {
51             System.out.println("Antrian kosong.");
52             return;
53         }
54
55         Pasien_2511533001 temp_3001 = head_3001;
56
57         System.out.println("\n== DAFTAR ANTRIAN ==");
58
59         while (temp_3001 != null) {
60             System.out.println(
61                 temp_3001.getNomorAntrian_3001()
62                 + "\n"
63                 + temp_3001.getNamaPasien_3001()
64                 + "\n"
65                 + temp_3001.getPenyakit_3001()
66             );
67             temp_3001 = temp_3001.next_3001;
68         }
69
70         // search
71         void cariPasien_3001(String nama_3001) {
72
73             Pasien_2511533001 temp_3001 = head_3001;
74
75             while (temp_3001 != null) {
76                 if (temp_3001.getNamaPasien_3001().equalsIgnoreCase(nama_3001)) {
77                     System.out.println("Pasien ditemukan!");
78                     System.out.println("No Antrian : " + temp_3001.getNomorAntrian_3001());
79                     return;
80                 }
81                 temp_3001 = temp_3001.next_3001;
82             }
83
84             System.out.println("Pasien tidak ditemukan.");
85         }
86
87         // status antrian
88         void cekStatus_3001() {
89
90             if (head_3001 == null) {

```

```

107         System.out.println("Antrian kosong.");
108         return;
109     }
110     System.out.println("Jumlah pasien : " + jumlah_3001);
111     System.out.println("Pasien terdepan : "
112         + head_3001.getNanaPasien_3001());
113 }
114
115 public static void main(String[] args) {
116     RumahSakit_2511533001 rs_3001 =
117         new RumahSakit_2511533001();
118
119     Scanner input_3001 = new Scanner(System.in);
120
121     int pilih_3001;
122
123     do {
124         System.out.println("\n=== Antrian Rumah Sakit NIM: 2511533001 ===");
125         System.out.println("1. Daftarkan Pasien (Insert)");
126         System.out.println("2. Panggil Pasien (Delete Head)");
127         System.out.println("3. Tampilkan Antrian (Display)");
128         System.out.println("4. Cari Pasien (Search)");
129         System.out.println("5. Cek Status Antrian");
130         System.out.println("6. Keluar");
131         System.out.print("Pilihan : ");
132
133         pilih_3001 = input_3001.nextInt();
134         input_3001.nextLine();
135
136         switch (pilih_3001) {
137             case 1:
138                 System.out.print("Masukkan nama pasien : ");
139                 String nama_3001 = input_3001.nextLine();
140
141                 System.out.print("Masukkan keluhan : ");
142                 String keluhan_3001 = input_3001.nextLine();
143
144                 rs_3001.daftarkanPasien_3001(nama_3001, keluhan_3001);
145                 break;
146             case 2:
147                 rs_3001.panggilPasien_3001();
148                 break;
149             case 3:
150                 rs_3001.tampilkanAntrian_3001();
151                 break;
152             case 4:
153                 System.out.print("Masukkan nama pasien : ");
154                 String cari_3001 = input_3001.nextLine();
155
156                 rs_3001.cariPasien_3001(cari_3001);
157                 break;
158             case 5:
159                 rs_3001.cekStatus_3001();
160
161                 break;
162             case 6:
163                 System.out.println("Program selesai.");
164                 break;
165             default:
166                 System.out.println("Pilihan tidak valid!");
167         }
168     } while (pilih_3001 != 6);
169
170     input_3001.close();
171 }
172
173 }
174
175 }
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196

```

## Penjelasan Program:

1. class RumahSakit\_2511533001 → Class ini digunakan untuk mengelola antrian pasien menggunakan struktur data Single Linked List (SLL).
2. Pasien\_2511533001 head\_3001, tail\_3001; → Variabel head\_3001 digunakan sebagai penunjuk node pertama, sedangkan tail\_3001 sebagai penunjuk node terakhir pada antrian pasien.
3. int nomor\_3001 = 0, jumlah\_3001 = 0; → Variabel nomor\_3001 digunakan untuk membuat nomor antrian otomatis dan jumlah\_3001 untuk menghitung jumlah pasien dalam antrian.
4. daftarkanPasien\_3001() → Method ini digunakan untuk menambahkan pasien baru ke dalam antrian rumah sakit.

5. `nomor_3001++;` → Nomor antrian akan bertambah setiap kali ada pasien baru yang didaftarkan.
6. `new Pasien_2511533001(...)` → Program membuat node pasien baru yang berisi nama pasien, keluhan, dan nomor antrian.
7. `if (head_3001 == null)` → Program mengecek apakah antrian masih kosong. Jika kosong, node baru menjadi head dan tail.
8. `tail_3001.next_3001 = baru_3001;` → Jika antrian tidak kosong, node baru ditambahkan di bagian belakang linked list.
9. `panggilPasien_3001()` → Method ini digunakan untuk memanggil sekaligus menghapus pasien paling depan pada antrian (delete head).
10. `head_3001 = head_3001.next_3001;` → Head dipindahkan ke node berikutnya sehingga pasien pertama keluar dari antrian.
11. `tampilkanAntrian_3001()` → Method ini digunakan untuk menampilkan seluruh daftar pasien yang ada dalam antrian.
12. `while (temp_3001 != null)` → Program melakukan traversal untuk menampilkan data pasien satu per satu sampai node terakhir.
13. `cariPasien_3001()` → Method ini digunakan untuk mencari data pasien berdasarkan nama pasien.
14. `equalsIgnoreCase()` → Digunakan untuk membandingkan nama pasien tanpa membedakan huruf besar dan kecil.
15. `cekStatus_3001()` → Method ini digunakan untuk menampilkan jumlah pasien dan pasien yang berada di posisi terdepan antrian.
16. Pada `main()` program menjalankan menu interaktif yang berisi fitur insert pasien, delete head, display antrian, search pasien, dan cek status antrian menggunakan input dari user.

## Output:

```
=== Antrian Rumah Sakit NIM: 2511533001 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan : 1
Masukkan nama pasien : Defan
Masukkan keluhan : tulang
Pasien berhasil didaftarkan!

=== Antrian Rumah Sakit NIM: 2511533001 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan : 1
Masukkan nama pasien : rayhan
Masukkan keluhan : pinggang
Pasien berhasil didaftarkan!

=== Antrian Rumah Sakit NIM: 2511533001 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan : 3

=== DAFTAR ANTRIAN ===
1. Defan - tulang
2. rayhan - pinggang

=== Antrian Rumah Sakit NIM: 2511533001 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
5. Keluar
Pilihan : 5
Jumlah pasien : 2
Pasien terdepan : Defan

=== Antrian Rumah Sakit NIM: 2511533001 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
5. Keluar
Pilihan : 4
Masukkan nama pasien : Defan
Pasien ditemukan!
No Antrian : 1

=== Antrian Rumah Sakit NIM: 2511533001 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan : 2

=== Pasien Dipanggil ===
No Antrian : 1
Nama : Defan
Keluhan : tulang

=== Antrian Rumah Sakit NIM: 2511533001 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan : 6
Program selesai.
```



## **BAB III**

### **PENUTUP**

#### **3.1 Kesimpulan**

Dari praktikum mengenai Single Linked List, dapat disimpulkan bahwa Single Linked List (SLL) merupakan struktur data linier yang terdiri dari kumpulan node yang saling terhubung melalui pointer. Setiap node menyimpan data dan referensi ke node berikutnya, sehingga data dapat dikelola secara dinamis tanpa harus disimpan secara berurutan seperti array.

Operasi dasar seperti penambahan node di depan, di belakang, maupun pada posisi tertentu, penghapusan node, traversal, dan pencarian data dapat dilakukan dengan baik pada Single Linked List. Struktur data ini lebih fleksibel dalam pengelolaan data karena tidak memerlukan pergeseran elemen saat terjadi penambahan atau penghapusan data.

Penerapan Single Linked List pada praktikum ini, seperti pada sistem antrian rumah sakit, menunjukkan bahwa konsep linked list dapat digunakan dalam pengolahan data secara terstruktur dan dinamis. Program mampu mengelola data pasien, menampilkan antrian, melakukan pencarian data, serta memanggil pasien sesuai urutan antrian.

Dengan demikian, Single Linked List tidak hanya penting dipahami secara teori, tetapi juga bermanfaat dalam menyelesaikan berbagai permasalahan pemrograman yang membutuhkan pengelolaan data secara fleksibel dan sistematis.

#### **3.2 Saran**

Sebaiknya dosen menyelenggarakan sesi pra-praktikum di kelas sebagai pengantar materi, agar mahasiswa memperoleh pemahaman awal yang lebih baik. Dengan demikian, potensi kepanikan atau kesalahan selama praktikum dapat diminimalkan. Disarankan agar materi praktikum dibagikan terlebih dahulu melalui platform iLearn, sehingga mahasiswa memiliki waktu yang cukup untuk mempelajari dan mempersiapkan diri sebelum pelaksanaan praktikum.

## **DAFTAR PUSTAKA**

[1] Oracle. "Single Linked List(Java Platform SE 17)". The Java™ Tutorials. 2024.

Available: <https://forums.oracle.com/ords/apexds/post/single-linked-list6324>